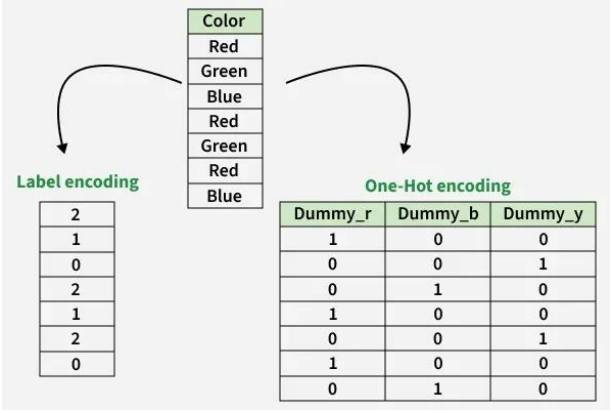
Q1. What is the difference between label encoding and one hot encoding technique? Why Use One-Hot Encoding Instead of Label Encoding?

1. Label Encoding technique: Label Encoding is a data preprocessing technique used to convert categorical values into numerical labels. It assigns a unique integer to each category, allowing machine learning algorithms to process categorical data in a numerical format. The encoded labels are typically assigned based on the sorted order of unique categories.



2. One Hot Encoding technique: One-Hot Encoding is another technique to convert categorical data into a numerical form. Instead of giving each category a single number, it creates a new binary column (1 or 0) for each category.

➢ Why Use One-Hot Encoding Instead of Label Encoding?

1 — Converting Categorical Data to Numerical Data: Machine learning models usually work with numerical data. Categorical data (like "Apple", "Chicken", "Broccoli") can't be used directly. One-Hot Encoding changes these categories into numbers. It makes the training process faster and more effective.

2 — Avoiding Category Ranking Problems: Label Encoding gives categories numerical values (e.g., Apple=1, Chicken=2, Broccoli=3). This can make the model think there is a ranking between the categories. One-Hot Encoding makes each category a separate column, avoiding this problem. In this way, the model can see each category as a separate column and understand the relationships between them correctly. This increases the model's accuracy and generalization ability.

3 — Better Performance and Accuracy: One-Hot Encoding helps the model learn each category independently, which can make the model more accurate.

Difference between label encoding and one hot encoding technique:

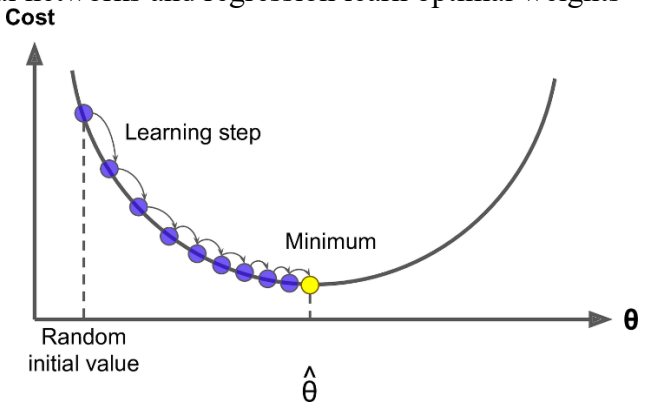
Column Names	One-Hot Encoding	Label Encoding
Description	Converts each unique category value into a new binary column.	Assigns each unique category value an integer.
Example	"India" -> [1, 0, 0] "Japan" -> [0, 1, 0] "USA" -> [0, 0, 1]	"India" -> 0 "Japan" -> 1 "USA" -> 2
When to Use	Non-ordinal categorical features. Manageable number of unique categories.	Ordinal categorical features. Large number of unique categories.
Advantages	Prevents the model from assuming any inherent order.	Simple and efficient for ordinal data. Does not increase dimensionality.
Disadvantages	Can lead to high dimensionality with many unique categories.	Imposes an arbitrary order on non-ordinal data. Model might assume false relationships.

Q2. What is Gradient Descent? What are the types of gradient descent?

What is Gradient Descent?

Gradient Descent is an iterative optimization algorithm used to minimize a cost function by adjusting model parameters in the direction of the steepest descent of the function's gradient. In simple terms, it finds the optimal values of weights and biases by gradually reducing the error between predicted and actual outputs.

- Minimizes the loss or error of a model
- Updates parameters step by step using gradients
- Helps models like neural networks and regression learn optimal weights



Formula:

This is the **Gradient Descent parameter update rule**.

You can write it as:

$$\theta = \theta - \alpha \nabla L(\theta)$$

Where:

- θ= model parameters (weights, coefficients, etc.)
- $L(\theta)$= loss (error) function
- $\nabla L(\theta)$= gradient of the loss function (direction of steepest increase)
- α= learning rate (step size)

Since our goal is to **minimize the error**, we move in the **opposite direction** of the gradient:

$$\text{New } \textit{Parameter} = \textit{Old Parameter} - \textit{Learning Rate} \times \textit{Gradient}$$

Different Variants of Gradient Descent

Types of gradient descent are:

1. **Batch Gradient Descent**: Batch Gradient Descent computes gradients using the entire dataset in each iteration. Batch gradient descent updates the model's parameters using the gradient of the entire training set. It calculates the average gradient of the cost function for all the training examples and updates the parameters in the opposite direction. Batch gradient descent guarantees convergence to the global minimum but can be computationally expensive and slow for large datasets.
2. **Stochastic Gradient Descent (SGD)**: SGD uses one data point per iteration to compute gradients, making it faster. Stochastic gradient descent updates the model's parameters using the gradient of one training example at a time. It randomly selects a training dataset example, computes the gradient of the cost function for that example, and updates the parameters in the opposite direction. Stochastic gradient descent is computationally efficient and can converge faster than batch gradient descent. However, it can be noisy and may not converge to the global minimum.
3. **Mini-batch Gradient Descent**: Mini-batch Gradient Descent combines batch and SGD by using small batches of data for updates. Mini-batch gradient descent updates the model's parameters using the gradient of a small batch size of the training dataset, known as a mini-batch. It calculates the average gradient of the cost function for the mini-batch and updates the parameters in the opposite direction. The mini-batch gradient descent algorithm combines the advantages of batch and stochastic gradient descent. It is the most commonly used method in practice. It is computationally efficient and less noisy than stochastic gradient descent while still being able to converge to a good solution.

Q3. What are the Challenges in Gradient Descent? How to solve them?

Despite being a powerful optimization algorithm, Gradient Descent faces several challenges that can impact its performance and convergence. Below are some of the key challenges:

1. Choosing the Right Learning Rate

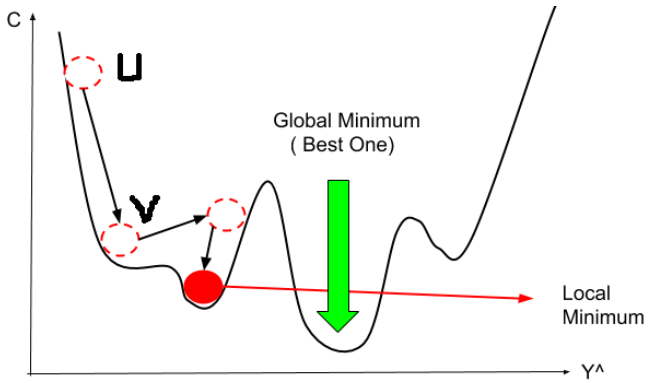
The learning rate (α) determines how large a step the algorithm takes toward the optimal solution.

- **If too small**: Convergence is extremely slow, requiring many iterations to reach the minimum.
- **If too large**: The algorithm may overshoot the minimum or diverge instead of converging.
- **Solution**: Techniques like learning rate scheduling, adaptive optimizers (e.g., Adam, RMSprop), or manual tuning help in selecting the right learning rate.

2. Local Minima and Saddle Points

Non-convex cost functions often have multiple local minima and saddle points, which can trap the algorithm.

- **Local minima**: The optimization process can get stuck in suboptimal points instead of reaching the global minimum.
- **Saddle points**: Points where the gradient is zero but are neither maxima nor minima, causing slow convergence.
- **Solution**: Advanced optimizers (e.g., Adam, Momentum) and proper weight initialization can help avoid these issues.



3. Feature Scaling

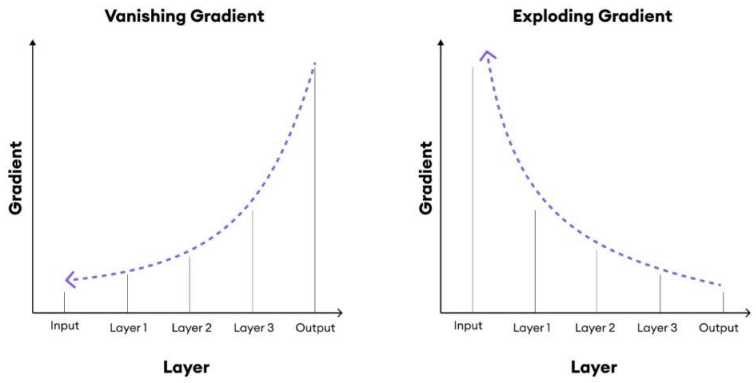
Gradient Descent works more efficiently when input features are scaled properly.

- **Unscaled features**: Features with vastly different magnitudes can lead to slower convergence as the algorithm struggles to adjust weights proportionally.
- **Solution**: **Standardization (zero mean, unit variance)** or **Normalization (scaling between 0 and 1)** ensures balanced updates, improving optimization speed.

4. Exploding or Vanishing Gradients

In deep neural networks, gradients can become too large (exploding) or too small (vanishing), making training unstable.

- **Exploding gradients**: Large gradient values cause excessive weight updates, leading to divergence.
- **Vanishing gradients**: Extremely small gradients slow down weight updates, preventing deep layers from learning effectively.
- **Solution**: Gradient clipping (to limit extreme values), weight initialization strategies (e.g., Xavier, He initialization), and activation functions like ReLU help mitigate this issue.



Q4. Explain the concept of bias-variance tradeoff. If you have trained a model which has a low bias and high variance, how would you deal with it?

Bias (Underfitting)

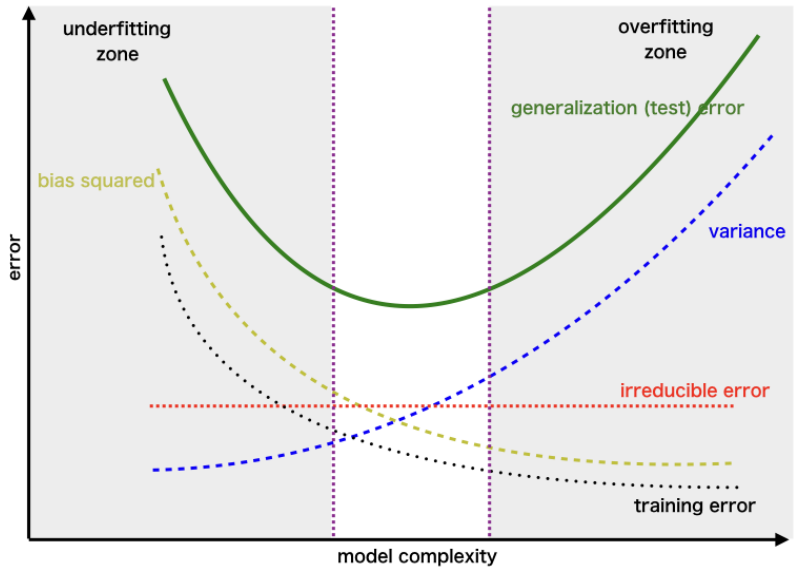
Bias happens when a model makes simplistic assumptions about the data. A model with high bias ignores important patterns and creates an overly simple view of the world.

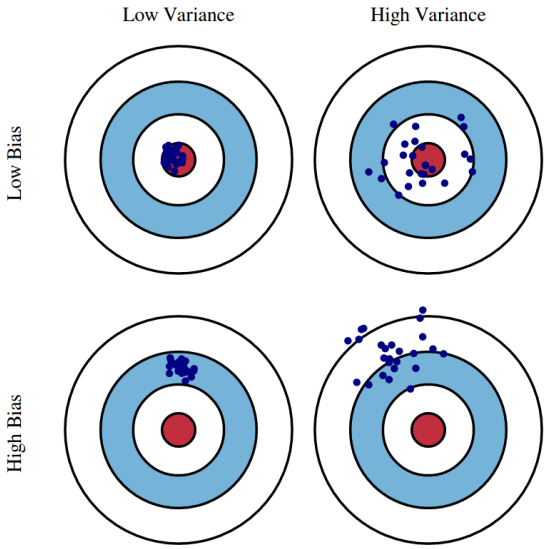
Variance (Overfitting)

Variance refers to how much a model's predictions would change if we trained it on different data. A model with high variance pays too much attention to the training data and captures not just patterns but also random noise.

The Visual Representation

Here's a simple way to visualize the bias-variance tradeoff:





The validation error curve shows bias and variance regions. The optimal model balances both error types.
In this curve:

- The left side (simple models) often has high bias/underfitting
- The right side (complex models) often has high variance/overfitting
- The sweet spot in the middle represents the optimal model complexity

Bias Variance Tradeoff

- Simple models usually have high bias and low variance, which may cause underfitting.
- Complex models usually have low bias but high variance, which may cause overfitting.
- Balanced model achieves an optimal point where both bias and variance are reasonably low.
- Goal of machine learning is to minimize the total prediction error on unseen data.

If you have trained a model which has a low bias and high variance, how would you deal with it?

A model with **low bias and high variance** is suffering from **overfitting**.

- **Low bias:** The model learns the training data very well, so training error is low.
- **High variance:** The model is sensitive to small changes in the training data, so it performs poorly on unseen/test data.

If a trained model has low bias and high variance, it is overfitting the training data. To reduce variance, we can collect more training data, simplify the model, apply regularization techniques (L1/L2), perform feature selection, use cross-validation, apply pruning in decision trees, use early stopping, and employ ensemble methods such as Random Forest. These techniques improve the model's ability to generalize to unseen data while maintaining acceptable bias.

Logistic Regression: Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values it predicts the probability that an input belongs to a specific class.

- It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It uses sigmoid function to convert inputs into a probability value between 0 and 1.

Working: Logistic regression computes a linear combination of input features ($z = w \cdot X + b$) and passes it through a sigmoid function to produce a probability between 0 and 1. This probability is then used to assign the input to a class.

Suppose we have input features represented as a matrix:

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1m} \\ x_{21} & \cdots & x_{2m} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nm} \end{bmatrix}$$

and the dependent variable is Y having only binary value i.e 0 or 1.

$$Y = \begin{cases} 0 & if\ Class1 \\ 1 & if\ Class2 \end{cases}$$

then, apply the multi-linear function to the input variables X.

$$z = \left(\sum_{i=1}^{n} w_i\, x_i\right) + b$$

Here x_i is the ith observation of X, $w_i = [w_1, w_2, w_3, \cdots, w_m]$ is the weights or Coefficient and b is the bias term also known as intercept. Simply this can be represented as the dot product of weight and bias.

$$z = w \cdot X + b$$

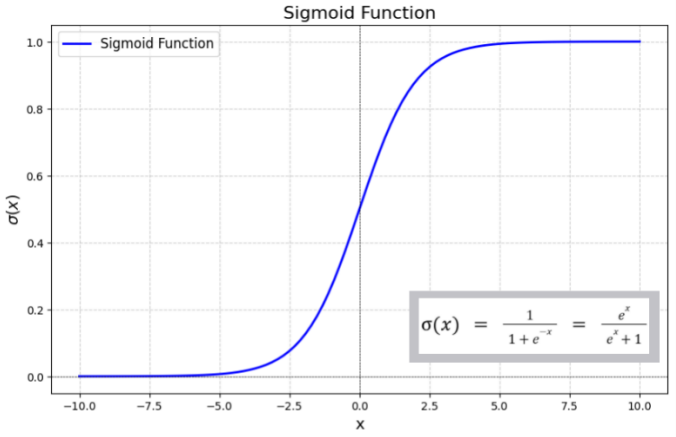
At this stage, z is a continuous value from the linear regression. Logistic regression then applies the sigmoid function to z to convert it into a probability between 0 and 1 which can be used to predict the class.

Sigmoid function is used as an activation function in machine learning and neural networks for modeling binary classification problems. Mathematically, sigmoid is represented as:

$$\sigma = \frac{1}{1 + e^{-x}}$$

where,

- x is the input value,
- e is Euler's number



Q5. The dataset of pass or fail in an exam of 5 students is given in the table. Use logistic regression as classifier to answer the following questions. After fitting the logistic regression model to the dataset, the estimated parameters are: $\beta_0 = -9.971, \beta_1 = 0.3596$

Hours Study (x)	Pass (y)
29	0
15	0
33	1
28	1
39	1

1. Calculate the probability of pass for the student who studied 33 hours.
2. At least how many hours student should study that makes he will pass the course with the probability of more than 95%.

Solution:

Step 1: Logistic Regression Model

The logistic regression hypothesis is

$$f(y) = \frac{1}{1 + e^{-z}}$$

$$f(y) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Given, estimated parameters are: $\beta_0 = -9.971,\ \beta_1 = 0.3596$

Therefore,

$$f(y) = \frac{1}{1 + e^{-(-9.971 + 0.3596x)}}$$

1. Probability of passing for a student who studied 33 hours

Substitute $x = 33$:

$$z = -9.971 + 0.3596(33) = 1.8979$$

Now,

$$f(y) = \frac{1}{1 + e^{-1.8979}}$$

$$f(y) \approx 0.8695$$

Answer:

$$if\ f(y) \geq 0.5, y = 1\ [Pass]$$

2. Minimum study hours required for probability of passing > 95%

We need

$$P(Pass) > 0.95$$

Using the logistic model:

$$0.95 = \frac{1}{1 + e^{-(-9.971 + 0.3596x)}}$$

Rearranging:

$$\ln\left(\frac{0.95}{0.05}\right) = -9.971 + 0.3596x$$

Since

$$2.9444 = -9.971 + 0.3596x$$

$$0.3596x = 12.9154$$

$$x = \frac{12.9154}{0.3596}$$

$$x \approx 35.91$$

Answer:

A student must study at least 36 hours to have a probability greater than 95% of passing.

Tutorial link → logistic regression solved example

Q6. What is Multicollinearity in Regression Analysis? How to mitigate it?

Multicollinearity occurs when two or more independent variables in a regression model are highly correlated with each other. In other words, multicollinearity exists when there are linear relationships among the independent variables, this causes issues in regression analysis because it does not follow the assumption of independence among predictors. Ex:

Age [x1]	Experience [x2]	Salary [y]
25	3	20k
30	8	30k
35	13	40k
40	18	50k
45	20	?

How to mitigate Multicollinearity in Regression Analysis?

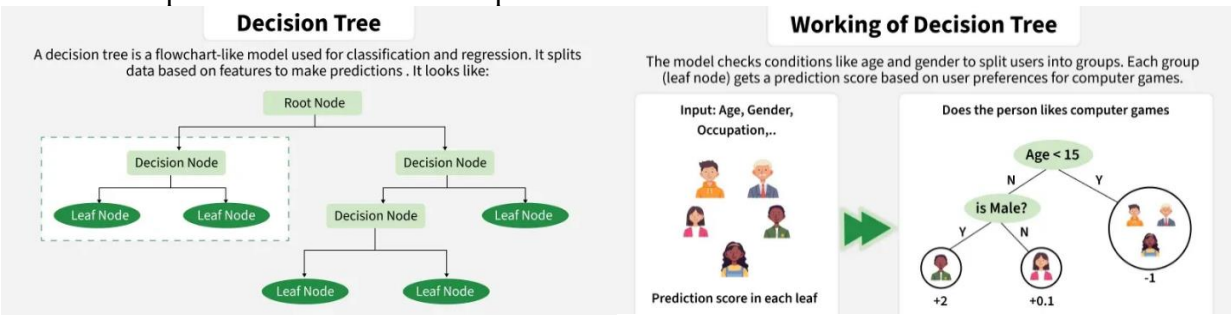
Mitigating multicollinearity in regression analysis is crucial for ensuring that your models provide reliable and interpretable results. Here are some effective strategies you can consider to address this issue:

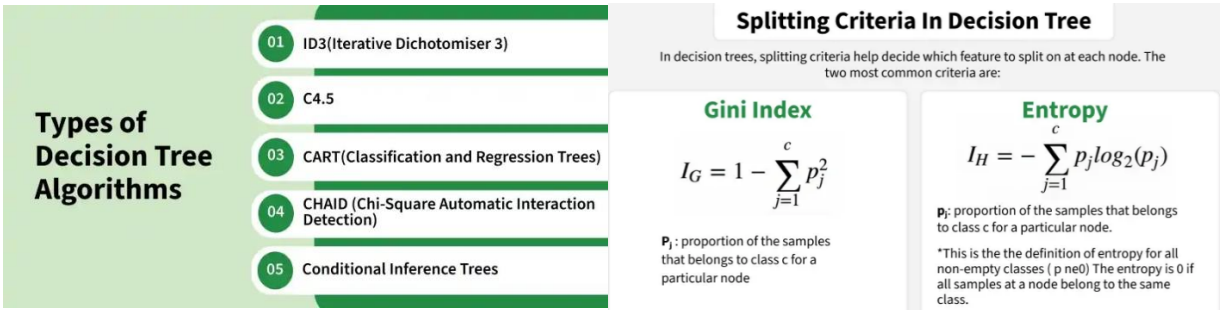
1. **Remove Highly Correlated Predictors**: Start by identifying and removing predictors that are highly correlated with each other. This can be determined through correlation matrices or Variance Inflation Factor (VIF) scores. Removing some of these variables can reduce multicollinearity without significantly impacting the model's accuracy.
2. **Principal Component Analysis (PCA)**: PCA can be used to transform the original correlated variables into a new set of uncorrelated variables (principal components). These principal components then serve as the predictors in your regression model. This technique is useful when you have many correlated variables.
3. **Ridge Regression**: This is a regularization method that introduces a penalty term (L2 norm) to the regression model. The penalty term is proportional to the square of the magnitude of the coefficients, which helps reduce their size and the impact of multicollinearity. Ridge regression is particularly useful when you want to keep all variables in the model but need to control for multicollinearity.

Q7. What is decision tree?

Decision Tree: A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart that helps in making step-by-step decisions, where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.





Steps to follow for decision tree problems:

1. Formula of entropy if a dataset contains:
   - p positive examples
   - n negative examples

   then

$$Entropy(S) = -\frac{p}{p+n}\log_2\left(\frac{p}{p+n}\right) - \frac{n}{p+n}\log_2\left(\frac{n}{p+n}\right)$$

2. Information Gain Formula

Information Gain measures how much uncertainty is reduced after splitting on an attribute.

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

where,

- S= original dataset
- A= attribute
- S_v =subset where attribute A has value v
- $|S_v|$ =number of records in subset
- $|S|$= total number of records

For the following dataset find the decision tree using ID3 algorithm:

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Solution:

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Outlook

$Values\,(Outlook) = Sunny, Overcast, Rain$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Sunny} \leftarrow [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.971$

$S_{Overcast} \leftarrow [4+, 0-]$ $\quad Entropy(S_{Overcast}) = -\frac{4}{4}log_2\frac{4}{4} - \frac{0}{4}log_2\frac{0}{4} = 0$

$S_{Rain} \leftarrow [3+, 2-]$ $\quad Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.971$

$$Gain\,(S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook) = Entropy(S) - \frac{5}{14}Entropy(S_{Sunny}) - \frac{4}{14}Entropy(S_{Overcast}) - \frac{5}{14}Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14}0.971 - \frac{4}{14}0 - \frac{5}{14}0.971 = 0.2464$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Temp

$Values\,(Temp) = Hot, Mild, Cool$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{Hot} \leftarrow [2+, 2-]$ $\quad Entropy(S_{Hot}) = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1.0$

$S_{Mild} \leftarrow [4+, 2-]$ $\quad Entropy(S_{Mild}) = -\frac{4}{6}log_2\frac{4}{6} - \frac{2}{6}log_2\frac{2}{6} = 0.9183$

$S_{Cool} \leftarrow [3+, 1-]$ $\quad Entropy(S_{Cool}) = -\frac{3}{4}log_2\frac{3}{4} - \frac{1}{4}log_2\frac{1}{4} = 0.8113$

$$Gain\,(S, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp) = Entropy(S) - \frac{4}{14}Entropy(S_{Hot}) - \frac{6}{14}Entropy(S_{Mild}) - \frac{4}{14}Entropy(S_{Cool})$$

$$Gain(S, Temp) = 0.94 - \frac{4}{14}1.0 - \frac{6}{14}0.9183 - \frac{4}{14}0.8113 = 0.0289$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Humidity

$Values\,(Humidity) = High, Normal$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$

$S_{High} \leftarrow [3+, 4-]$ $\quad Entropy(S_{High}) = -\frac{3}{7}log_2\frac{3}{7} - \frac{4}{7}log_2\frac{4}{7} = 0.9852$

$S_{Normal} \leftarrow [6+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{6}{7}log_2\frac{6}{7} - \frac{1}{7}log_2\frac{1}{7} = 0.5916$

$$Gain\,(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity) = Entropy(S) - \frac{7}{14}Entropy(S_{High}) - \frac{7}{14}Entropy(S_{Normal})$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14}0.9852 - \frac{7}{14}0.5916 = 0.1516$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute: Wind

$Values\ (Wind) = Strong, Weak$

$S = [9+, 5-]$ $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

$S_{Strong} \leftarrow [3+, 3-]$ $\quad Entropy(S_{Strong}) = 1.0$

$S_{Weak} \leftarrow [6+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{6}{8} log_2 \frac{6}{8} - \frac{2}{8} log_2 \frac{2}{8} = 0.8113$

$$Gain\ (S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = Entropy(S) - \frac{6}{14} Entropy(S_{Strong}) - \frac{8}{14} Entropy(S_{Weak})$$

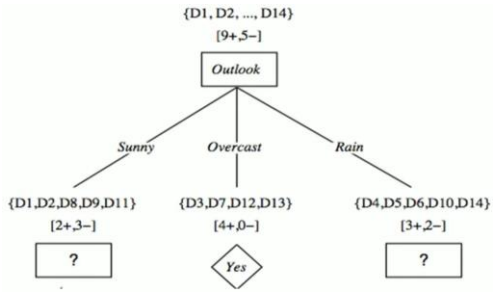
$$Gain(S, Wind) = 0.94 - \frac{6}{14}\ 1.0 - \frac{8}{14}\ 0.8113 = 0.0478$$

$Gain(S, Outlook) = 0.2464$

$Gain(S, Temp) = 0.0289$

$Gain(S, Humidity) = 0.1516$

$Gain(S, Wind) = 0.0478$



Day	Temp	Humidity	Wind	Play Tennis
D1	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
D11	Mild	Normal	Strong	Yes

Attribute: Temp

$Values\ (Temp) = Hot, Mild, Cool$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5} log_2 \frac{2}{5} - \frac{3}{5} log_2 \frac{3}{5} = 0.97$

$S_{Hot} \leftarrow [0+, 2-]$ $\quad Entropy(S_{Hot}) = 0.0$

$S_{Mild} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Mild}) = 1.0$

$S_{Cool} \leftarrow [1+, 0-]$ $\quad Entropy(S_{Cool}) = 0.0$

$$Gain\ (S_{Sunny}, Temp) = Entropy(S) - \sum_{v \in \{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Temp)$$

$$= Entropy(S) - \frac{2}{5} Entropy(S_{Hot}) - \frac{2}{5} Entropy(S_{Mild})$$

$$- \frac{1}{5} Entropy(S_{Cool})$$

$$Gain(S_{sunny}, Temp) = 0.97 - \frac{2}{5} 0.0 - \frac{2}{5}\ 1 - \frac{1}{5} 0.0 = 0.570$$

Day	Temp	Humidity	Wind	Play Tennis
Dl	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
Dl1	Mild	Normal	Strong	Yes

Attribute: Humidity

$Values\ (Humidity) = High, Normal$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S) = -\frac{2}{5} log_2 \frac{2}{5} - \frac{3}{5} log_2 \frac{3}{5} = 0.97$

$S_{high} \leftarrow [0+, 3-]$ $\quad Entropy(S_{High}) = 0.0$

$S_{Normal} \leftarrow [2+, 0-]$ $\quad Entropy(S_{Normal}) = 0.0$

$$Gain\ (S_{Sunny}, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Sunny}, Humidity) = Entropy(S) - \frac{3}{5} Entropy(S_{High}) - \frac{2}{5} Entropy(S_{Normal})$$

$$Gain(S_{sunny}, Humidity) = 0.97 - \frac{3}{5}\ 0.0 - \frac{2}{5} 0.0 = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
DI	Hot	High	Weak	No
D2	Hot	High	Strong	No
D8	Mild	High	Weak	No
D9	Cool	Normal	Weak	Yes
DI1	Mild	Normal	Strong	Yes

Attribute: Wind

$Values\ (Wind) = Strong, Weak$

$S_{Sunny} = [2+, 3-]$ $\quad Entropy(S) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.97$

$S_{Strong} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Strong}) = 1.0$

$S_{Weak} \leftarrow [1+, 2-]$ $\quad Entropy(S_{Weak}) = -\frac{1}{3}log_2\frac{1}{3} - \frac{2}{3}log_2\frac{2}{3} = 0.9183$

$$Gain\ (S_{Sunny}, Wind) = Entropy(S) - \sum_{v\ \in\{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

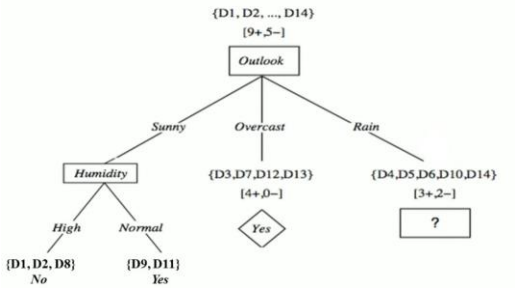
$$Gain(S_{Sunny}, Wind) = Entropy(S) - \frac{2}{5}Entropy(S_{Strong}) - \frac{3}{5}Entropy(S_{Weak})$$

$$Gain(S_{sunny}, Wind) = 0.97 - \frac{2}{5}1.0 - \frac{3}{5}\ 0.918 = 0.0192$$

$Gain(S_{sunny}, Temp) = 0.570$

$Gain(S_{sunny}, Humidity) = 0.97$

$Gain(S_{sunny}, Wind) = 0.0192$



Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

Attribute: Temp

$Values\ (Temp) = Hot, Mild, Cool$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{Hot} \leftarrow [0+, 0-]$ $\quad Entropy(S_{Hot}) = 0.0$

$S_{Mild} \leftarrow [2+, 1-]$ $\quad Entropy(S_{Mild}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$

$S_{Cool} \leftarrow [1+, 1-]$ $\quad Entropy(S_{Cool}) = 1.0$

$$Gain\ (S_{Rain}, Temp) = Entropy(S) - \sum_{v\ \in\{Hot, Mild, Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp)$$

$$= Entropy(S) - \frac{0}{5}Entropy(S_{Hot}) - \frac{3}{5}Entropy(S_{Mild})$$

$$- \frac{2}{5}Entropy(S_{Cool})$$

$$Gain(S_{Rain}, Temp) = 0.97 - \frac{0}{5}0.0 - \frac{3}{5}\ 0.918 - \frac{2}{5}\ 1.0 = 0.0192$$

Attribute: Humidity

$Values\ (Humidity) = High, Normal$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{High} \leftarrow [1+, 1-]$ $\quad Entropy(S_{High}) = 1.0$

$S_{Normal} \leftarrow [2+, 1-]$ $\quad Entropy(S_{Normal}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$

$$Gain\ (S_{Rain}, Humidity) = Entropy(S) - \sum_{v\ \in\{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = Entropy(S) - \frac{2}{5}Entropy(S_{High}) - \frac{3}{5}Entropy(S_{Normal})$$

$$Gain(S_{Rain}, Humidity) = 0.97 - \frac{2}{5}\ 1.0 - \frac{3}{5}0.918 = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
Dl0	Mild	Normal	Weak	Yes
Dl4	Mild	High	Strong	No

Attribute: Wind

$Values\ (wind) = Strong, Weak$

$S_{Rain} = [3+, 2-]$ $\quad Entropy(S_{Sunny}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$

$S_{Strong} \leftarrow [0+, 2-]$ $\quad Entropy(S_{Strong}) = 0.0$

$S_{Weak} \leftarrow [3+, 0-]$ $\quad Entropy(S_{weak}) = 0.0$

$$Gain\ (S_{Rain}, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

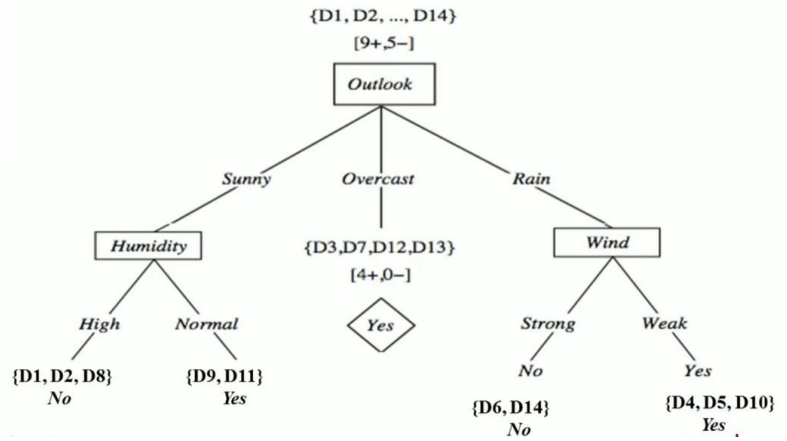
$$Gain(S_{Rain}, Wind) = Entropy(S) - \frac{2}{5}Entropy(S_{Strong}) - \frac{3}{5}Entropy(S_{Weak})$$

$$Gain(S_{Rain}, Wind) = 0.97 - \frac{2}{5}\,0.0 - \frac{3}{5}0.0 = 0.97$$

$Gain(S_{Rain}, Temp) = 0.0192$

$Gain(S_{Rain}, Humidity) = 0.0192$

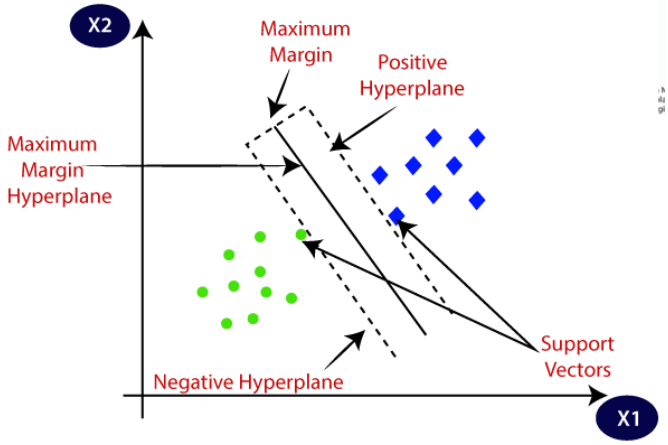
$Gain(S_{Rain}, Wind) = 0.97$



Tutorial Link → [Decision Tree Solved Numerical]

Q8. What is Support Vector Machine? How SVM handles the data when they are not linearly separable?

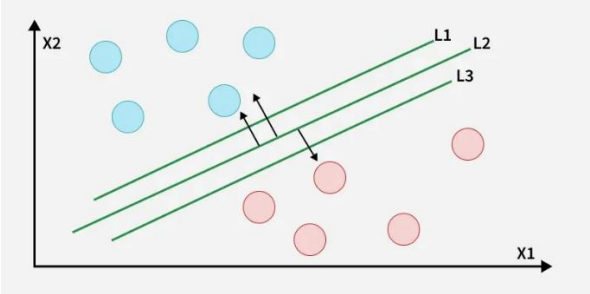
Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.



The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

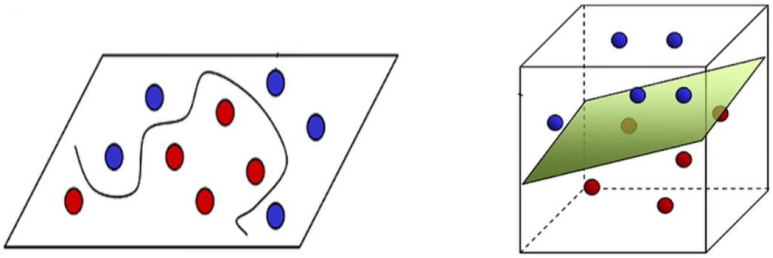
Working of Support Vector Machine Algorithm

The key idea behind the SVM algorithm is to find the hyperplane that best separates two classes by maximizing the margin between them. This margin is the distance from the hyperplane to the nearest data points (support vectors) on each side.



The best hyperplane also known as the hard margin is the one that maximizes the distance between the hyperplane and the nearest data points from both classes. This ensures a clear separation between the classes. So, from the above figure, we choose L2 as hard margin.

When data is not linearly separable i.e it can't be divided by a straight line, SVM uses a technique called kernels to map the data into a higher-dimensional space where it becomes separable. This transformation helps SVM find a decision boundary even for non-linear data.

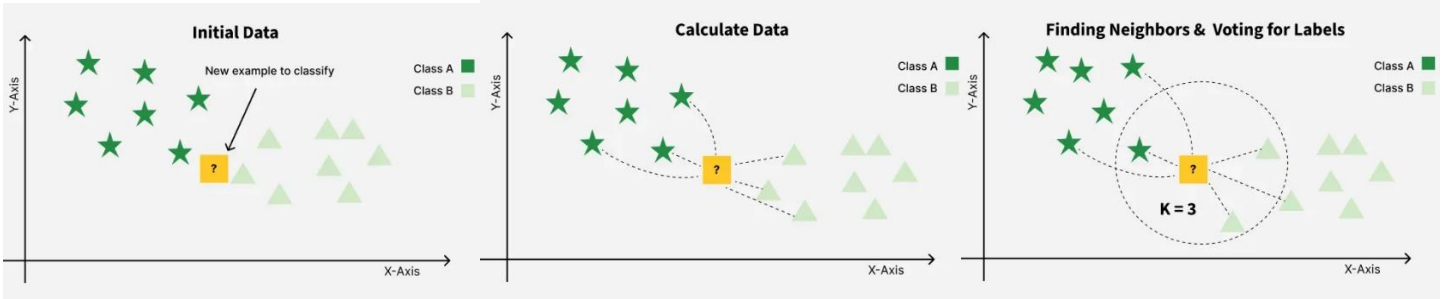


A kernel is a function that maps data points into a higher-dimensional space without explicitly computing the coordinates in that space. This allows SVM to work efficiently with non-linear data by implicitly performing the mapping. For example, consider data points that are not linearly separable. By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable.

Q9. K-Nearest Neighbor (KNN) Algorithm in machine learning.

K-Nearest Neighbor (KNN) is a simple and widely used machine learning technique for classification and regression tasks. It works by identifying the *K* closest data points to a given input and making predictions based on the majority class or average value of those neighbors.

- Classifies data based on similarity with nearby data points
- Uses distance metrics like Euclidean distance to find nearest neighbors
- Since KNN makes no assumptions about the underlying data distribution, it makes it a non-parametric and instance-based learning method.



KNN is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.

Q10. Solve the following problem using KNN algorithm when K = 5.

Sepal Length	Sepal Width	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3.0	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Virginica
6.1	2.8	Verscicolor
7.3	2.9	Virginica
6.0	2.7	Verscicolor
5.8	2.8	Virginica
6.3	2.3	Verscicolor
5.1	2.5	Verscicolor
6.3	2.5	Verscicolor
5.5	2.4	Verscicolor

Step 1: Find Distance

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(x-a)^2 + (y-b)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = \sqrt{(5.2-5.3)^2 + (3.1-3.7)^2}$$

$$\text{Distance (Sepal Length, Sepal Width)} = 0.608$$

Sepal Length	Sepal Width	Species	Distance
5.3	3.7	Setosa	0.608

Sepal Length	Sepal Width	Species
5.2	3.1	?

Sepal Length	Sepal Width	Species	Distance	Rank
5.3	3.7	Setosa	0.608	3
5.1	3.8	Setosa	0.707	6
7.2	3.0	Virginica	2.002	13
5.4	3.4	Setosa	0.36	2
5.1	3.3	Setosa	0.22	1
5.4	3.9	Setosa	0.82	8
7.4	2.8	Virginica	2.22	15
6.1	2.8	Verscicolor	0.94	10
7.3	2.9	Virginica	2.1	14
6.0	2.7	Verscicolor	0.89	9
5.8	2.8	Virginica	0.67	5
6.3	2.3	Verscicolor	1.36	12
5.1	2.5	Verscicolor	0.60	4
6.3	2.5	Verscicolor	1.25	11
5.5	2.4	Verscicolor	0.75	7

Step 3: Find the Nearest Neighbor

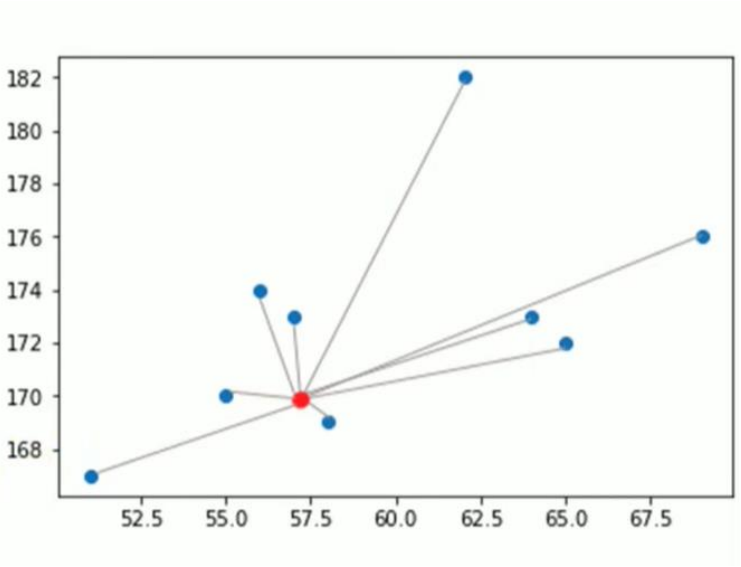
If k = 1 – Setosa

If k = 2 – Setosa

If k = 5 – Setosa

Q11. Solve the following problem using KNN algorithm when K = 5.

Height (CM)	Weight (KG)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	?



Solution:

Height (CM)	Weight (KG)	Class	Distance
167	51	Underweight	6.7
182	62	Normal	13
176	69	Normal	13.4
173	64	Normal	7.6
172	65	Normal	8.2
174	56	Underweight	4.1
169	58	Normal	1.4
173	57	Normal	3
170	55	Normal	2
170	57	?	

THE DISTANCE FORMULA

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Height (CM)	Weight (KG)	Class	Distance	Rank
169	58	Normal	1.4	1
170	55	Normal	2	2
173	57	Normal	3	3
174	56	Underweight	4.1	4
167	51	Underweight	6.7	5
173	64	Normal	7.6	6
172	65	Normal	8.2	7
182	62	Normal	13	8
176	69	Normal	13.4	9
170	57	?		

If K=1, Normal

If K=2, Normal

If K=3, Normal

If K=4, Normal

If K=5, Normal

Q12. Solve the following problem using KNN algorithm when K = 3.

KNN Classification Numerical Example To solve the numerical example on the K-nearest neighbor i.e. KNN classification algorithm, we will use the following dataset.

Point	Coordinates	Class Label
A1	(2,10)	C2
A2	(2, 6)	C1
A3	(11,11)	C3
A4	(6, 9)	C2
A5	(6, 5)	C1
A6	(1, 2)	C1
A7	(5, 10)	C2
A8	(4, 9)	C2
A9	(10, 12)	C3
A10	(7, 5)	C1
A11	(9, 11)	C3
A12	(4, 6)	C1
A13	(3, 10)	C2
A15	(3, 8)	C2
A15	(6, 11)	C2

KNN classification dataset

In the above dataset, we have fifteen data points with three class labels. Now, suppose that we have to find the class label of the point P= (5, 7).

Solution:

For this, we will first specify the number of nearest neighbors i.e. k. Let us take k to be 3.

Now, we will find the distance of P to each data point in the dataset. For this KNN classification numerical example, we will use the Euclidean distance metric. The following table shows the Euclidean distance of P to each data point in the dataset.

Point	Coordinates	Distance from P (5, 7)
A1	(2, 10)	4.24
A2	(2, 6)	3.16
A3	(11, 11)	7.21
A4	(6, 9)	2.23
A5	(6, 5)	2.23
A6	(1, 2)	6.40
A7	(5, 10)	3.0
A8	(4, 9)	2.23
A9	(10, 12)	7.07
A10	(7, 5)	2.82
A11	(9, 11)	5.65
A12	(4, 6)	1.41
A13	(3, 10)	3.60
A15	(3, 8)	2.23
A15	(6, 11)	4.12

Distance of each point for the new point

After finding the distance of each point in the dataset to P, we will sort the above points according to their distance from P (5, 7). After sorting, we get the following table.

Point	Coordinates	Distance from P (5, 7)
A12	(4, 6)	1.41
A4	(6, 9)	2.23
A5	(6, 5)	2.23
A8	(4, 9)	2.23
A15	(3, 8)	2.23
A10	(7, 5)	2.82
A7	(5, 10)	3
A2	(2, 6)	3.16
A13	(3, 10)	3.6
A15	(6, 11)	4.12
A1	(2, 10)	4.24
A11	(9, 11)	5.65
A6	(1, 2)	6.4
A9	(10, 12)	7.07
A3	(11, 11)	7.21

Sorted Points according to distance

- ✓ As we have taken k=3, we will now consider the class labels of three points in the dataset nearest to point P to classify P In the above table, A12, A4, and A5 are the closest 3 neighbors of point P.
- ✓ Hence, we will use the class labels of points A12, A4, and A5 to decide the class label for P.
- ✓ Now, point A12, A4, and A5 have the class labels C1, C2, and C1 respectively. Among these points, the majority class label is C1.
- ✓ Therefore, we will specify the class label of point P = (5, 7) as C1. Hence, we have successfully used KNN classification to classify point P according to the given dataset.